

# 비개발자 유지보수를 위한 준비 — 공통사항

## 1) 현재 스키마에서 확장성을 해칠 수 있는 부분이 있나요?

있습니다. 다만 “지금 당장 망한다”가 아니라, 성장하면 운영/리포트/품질관리에서 발목 잡힐 부분이 몇 가지 있어요.

### 확장성을 해치는 대표 패턴

#### 1. JSONB/배열 컬럼에 핵심 데이터를 넣는 구조

- 초반엔 빠르지만, 나중에 “통계/검색/리포트/필터”가 늘어나면 쿼리가 급격히 어려워지고 성능 튜닝도 힘들어집니다.
- 예: 체크리스트 항목, 품 필드 정의, 활동 점수, 코드 리스트 등  
→ 성장하면 **행(row) 단위로 정규화**하는 게 안전합니다.

#### 1. 상태/카운트 같은 값의 “중복 저장”

- 예: 예약 테이블에 `student_count`, `payment_status` 같은 값이 있고, 실제 원본 데이터는 다른 테이블(참석자/결제)에 있는 경우.
- 이런 구조는 “쓰기 성능/화면 속도”는 좋아지지만, 관리가 느슨하면 **불일치 데이터가 쌓여 서비스 신뢰도가 깨집니다.**  
→ “단일 진실(Source of Truth)”을 정하거나, “캐시”로 인정하고 **동기화 로직(트리거/서버 코드)**을 명확히 해야 합니다.

#### 1. 폴리모픽 참조(reference\_type + reference\_id)

- 포인트 거래, 이벤트 로그에서 흔히 쓰는데 FK를 걸 수 없어 “고아 데이터”가 생깁니다.  
→ 운영 안정성을 위해 “최소한 reference\_type 값 제한 + 검증 로직”이 필요합니다.

정리:

- MVP 단계에서는 괜찮지만, **추천/통계/운영 자동화가 커질수록 JSONB/배열·중복상태 저장·폴리모픽 참조가 “비개발자 유지보수”를 어렵게 만들어요.**

## 2) 예약 시스템의 동시성 제어가 적절한가요?

지금 방식(예: slot lock 테이블로 선점) 자체는 방향이 맞습니다.

다만 "동시성은 설계는 맞아도, 디테일에서 실제 장애가 많이 납니다."

## 체크해야 할 동시성 포인트 (실무 핵심)

### 1. "시간 겹침"을 완전히 막는가?

- 단순히 시작시간이 같은 것만 막으면,  
10:00-11:00 예약이 있는데 10:30-11:30 예약이 들어오는 문제가 생길 수 있습니다.
- 진짜 안전하게 하려면 DB 레벨에서 "겹침 금지"가 필요합니다.
  - PostgreSQL에서는 `tstzrange` + `EXCLUDE USING gist` 가 정석입니다.

### 1. 락(선점) 만료 처리

- `expires_at` 이 지나면 자동 삭제/정리가 되어야 합니다.
- `pg_cron`을 못 쓰는 환경(로컬/일부 운영)도 있으니,
  - 서버 배치(서버 cron) / Supabase scheduled job / 외부 스케줄러 등  
정리 주체를 하나 정해서 운영해야 합니다.

### 1. 트랜잭션 경계

- "선점 → 결제 → 예약 확정" 과정에서 트랜잭션이 깨지면
  - 선점은 남고 예약은 안 생김(유령 락)
  - 예약은 생겼는데 결제 실패 상태 반영이 안 됨
- 서버 코드에서 트랜잭션/재시도 정책이 있어야 합니다.

정리:

- 지금 구조는 출발이 좋지만, "겹침 방지(기간 충돌)" + "락 만료 운영" + "트랜잭션 일관성"을 확실히 잡아야 실제 운영에서 안전합니다.

---

## 3) 개인정보 관리(이름, 생년월일, 주소, 아이 정보)를 위한 추가 보안 조치가 필요한가요?

네, 필요한 수준입니다. 특히 "아이 정보"는 법/신뢰 관점에서 민감도가 훨씬 높습니다.

Supabase는 기본 보안이 있어도 설계/운영을 잘못하면 대시보드 수정/권한/로그에서 유출이 납니다.

## DBA/운영 관점 권장 조치

## 1. RLS(Row Level Security) + 최소 권한

- 기본은 "사용자 본인/해당 호스트만 조회 가능"
- 관리자도 "필요한 화면만" 접근하도록 역할 분리

### 1. 민감 필드 분리

- 추천: `profiles` 같은 일반 프로필 테이블과 `pii_private` 같은 민감정보 테이블을 분리하고 접근 정책을 더 강하게
- 대시보드에서 실수로 노출/수정되는 위험이 줄어듭니다.

### 1. 암호화(선택)

- 주소/생년월일은 "서비스 목적상 평문 조회가 꼭 필요한지"부터 따져야 합니다.
- 필요 없다면:
  - 생년월일은 "연령대"로 변환해 저장(예: 30대/40대)
  - 주소는 "시/구"만 저장하고 상세주소는 별도 보관(또는 암호화)
- 꼭 평문이 필요하다면:
  - 애플리케이션 레벨 암호화(키 관리) 또는
  - DB 암호화(환경 지원 여부 확인)

### 1. 접근 로그

- 누가 언제 어떤 PII를 조회/수정했는지 로그가 남아야 합니다.
- 최소: 관리자 화면에서 PII 조회 시 `admin_action_logs`에 남기기

정리:

- 아이 정보가 들어간 순간부터는 "RLS + 민감 테이블 분리 + 접근로그"는 사실상 필수입니다.

---

## 4) 비개발자가 직접 수정하면 안 되는 테이블/필드는 무엇인가요?

비개발자(운영자)가 Supabase 대시보드에서 직접 만지면 서비스가 깨지거나 돈/정산/예약 꼬이는 구간을 정리하면 아래입니다.

### 절대 직접 수정 금지(원칙)

#### 1. 결제/정산 관련

- payments, refunds, platform\_fees, host\_credits, point\_transactions 등
- 이유: 회계 데이터는 "변경 이력/정합성"이 생명  
→ 수정이 필요하면 "정정 트랜잭션(추가 row)"로 처리해야 합니다.

### 1. 예약 핵심 상태

- bookings.status / schedule 시간 / 참석자 테이블
- 락/예약/결제가 연결되어 있어 한 필드만 바꾸면 연쇄적으로 오류

### 1. RLS/권한 정책 관련

- policies, auth 매핑, role 등
- 한 번 잘못 건드리면 전체 데이터가 공개되거나 서비스가 작동 안 함

### 1. 참조 무결성의 키(FK)

- 모든 `_id` FK 값 수동 변경 금지  
→ 연결이 깨지면 복구가 매우 어렵습니다.

## "조건부 수정 가능(운영 가이드 필요)"

- products(상품): 가격/설명 수정은 가능하지만, ID/매핑/소스키는 금지
- coupons: 할인정책 변경은 가능하나, 사용된 쿠폰/거래내역은 금지
- notifications: 일부 메시지 삭제는 가능하나 감사 목적이면 보관 권장

### 운영 규칙 한 줄 요약:

|"돈(결제/정산)·시간(예약)·권한(RLS)"은 운영자가 직접 수정하지 않는다.

## 5) 정기적으로 모니터링해야 할 쿼리나 메트릭이 있나요?

네. 장애는 대부분 '느려짐 + 누락 + 불일치'로 시작합니다.

사업 운영에서 최소로 봐야 할 것만 뽑으면:

### A. 예약/락 관련

- 만료된 락 수(정리 안 된 선점)
- 같은 시간대 중복 예약 여부(논리 오류 탐지)
- 예약 생성 실패율(결제 성공 대비 예약 확정 누락)

## B. 결제/정산 관련

- 결제 성공/실패 비율
- 환불 처리 지연 건수
- 정산 합계 불일치(예: payment.amount vs payout+fee)

## C. 성능(느려짐)

- 느린 쿼리 Top N (Supabase Log/DB 통계)
- 테이블 부하: bookings, chat\_messages, conversion\_events 같은 “급성장 테이블” row 증가
- 인덱스가 안 타는 조회(특정 목록 화면이 느려지는 경우)

## D. 보안/권한

- RLS 정책 오류로 인한 접근 실패(403 급증)
- 관리자 계정의 PII 조회 횟수/이상 패턴

---

# Supabase 대시보드에서 직접 데이터 수정할 때 주의사항

### 1. 반드시 “작업 전 스냅샷”

- 수정하려는 row를 CSV로 내려받거나,
- 수정 대상 PK 목록을 저장해둡니다.

### 1. FK/UUID는 건드리지 않는다

- id, user\_id, booking\_id 같은 키를 바꾸면 복구가 어렵습니다.

### 1. 결제/정산/포인트는 “수정”이 아니라 “정정 row 추가”

- 회계성 데이터는 감사/분쟁 이슈가 있어서 “기존 row 덮어쓰기”는 금지

### 1. 운영 DB에서 대량 수정 금지

- 대량 UPDATE/DELETE는 락/성능 이슈 발생 가능  
→ 반드시 제한된 범위 + 사전 SELECT로 검증

### 1. RLS 환경에서 ‘대시보드로 보이는 것’이 곧 전체가 아니다

- 대시보드는 서비스 계정 권한으로 보이는 경우가 많아,

앱에서 보이는 범위와 다를 수 있습니다.

---

## 백업 전략 (Supabase 자동 백업되지만 확인 필요)

자동 백업이 있어도 “복구가 가능한지”를 확인하지 않으면 의미가 없습니다.

### 최소 체크리스트

#### 1. 백업 주기/보관 기간 확인

- 자동 백업이 며칠치인지(예: 7일/30일)
- PITR(Point-In-Time Recovery) 가능 여부(플랜에 따라 다름)

#### 1. 복구 시나리오를 미리 정한다

- “테이블 하나만 복구?”
  - “특정 시점으로 전체 롤백?”
  - “부분 데이터만 되돌리기?”
- 이게 정리되어야 사고 때 결정이 빨라집니다.

#### 1. 운영에선 ‘스키마 변경’도 백업 대상

- DDL 변경(테이블/컬럼/정책)은 Git에 스크립트로 남겨야 합니다.
- 비개발자라도 “스키마 변경 스크립트”를 저장해두면 복구력이 확 올라갑니다.

---

## 로그 모니터링 포인트 (현업에서 실제로 보는 것)

#### 1. 에러율

- 5xx 증가
- 결제 API 실패율 증가
- 예약 생성 실패(특히 동시성 구간)

#### 1. 403/권한 관련

- 갑자기 403이 급증하면 RLS 정책 변경/버그 가능성 높음

#### 1. 특정 API의 응답시간 상승

- “예약 조회 화면”, “추천 목록 화면”이 1~2초 넘어가면 바로 사용자 이탈

## 1. 배치/정리 작업 실패

- 만료 락 삭제 실패
  - 크롤링 실패율 증가(차단/파서 깨짐)
  - 데이터 폭증(스냅샷 테이블 급격히 증가)
- 

# 마지막으로, 비개발자 운영을 위한 "한 장짜리 원칙"

- 돈/시간/권한은 직접 수정 금지
- 변경은 항상 기록(로그) 남기기
- 대량 작업은 절대 운영 DB에서 즉흥적으로 하지 않기
- 느려지기 전에 Top 느린 쿼리/테이블 증가를 주기적으로 확인